

PROGRAM

```
#include<stdio.h>
#include<stdlib.h>
//Define the structure for a node
struct Node{
    int data;
    struct Node* next;
};
//Function prototypes
struct Node* createNode(int data);
void insertAtEnd(struct Node** head, int data);
void insertAtPosition(struct Node** head, int data, int position);
int searchElement(struct Node** head, int data);
void deleteElement(struct Node** head, int position);
void displayList(struct Node* head);
int main() {
    struct Node* head = NULL;
    int choice, data, position, result;
    while(1) {
        printf("\nSingly Linked List Operations Menu:\n");
        printf("1. Insert at End\n");
        printf("2. Insert at Position\n");
        printf("3. Search Element\n");
        printf("4. Delete Element\n");
        printf("5. Display List\n");
        printf("6. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch(choice) {
            case 1:
                printf("Enter data to insert at end: ");
                scanf("%d", &data);
                insertAtEnd(&head, data);
                break;
            case 2:
                printf("Enter data to insert: ");
                scanf("%d", &data);
                printf("Enter position to insert (0 – based index): ");
                scanf("%d", &position);
                insertAtPosition(&head, data, position);
                break;
            case 3:
                printf("Enter element to search: ");
                scanf("%d", &data);
                result = searchElement(&head, data);
                if(result != -1)
                    printf("Element found at position: %d\n", result);
                else
                    printf("Element not found in the list\n");
                break;
```

```

        case 4:
            printf("Enter position to delete (0 – based index): ");
            scanf("%d", &position);
            deleteElement(&head, position);
            break;
        case 5:
            displayList(head);
            break;
        case 6:
            //Free memory allocated for the linked list before exit
            while(head != NULL) {
                struct Node* temp = head;
                head = head->next;
                free(temp);
            }
            return 0;
        default:
            printf("Invalid choice! Please enter a valid option.\n");
    }
}
return 0;
}

// Function to create a new node
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    if(newNode == NULL) {
        fprintf(stderr, "Memory allocation failed\n");
        exit(EXIT_FAILURE);
    }
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}

// Function to insert a note at the end of the list
void insertAtEnd(struct Node** head, int data) {
    struct Node* newNode = createNode(data);
    if(*head == NULL) {
        *head = newNode;
        return;
    }
    struct Node* temp = *head;
    while(temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = newNode;
}

//Function to insert a node at a specified position
void insertAtPosition(struct Node** head, int data, int position)
{
    if(position == 0) {
        newNode->next = *head;
    }
}

```

```

        *head = newNode;
        return;
    }
    struct Node* current = *head;
    struct Node* previous = NULL;
    int currentPos = 0;
    while((current != NULL) && (currentPos < position)) {
        previous = current;
        current = current->next;
        currentPos++;
    }
    if(currentPos != position) {
        printf("Error: Invalid position.\n");
        free(newNode);
        return;
    }
    newNode->next = current;
    previous->next = newNode;
}
//Function to search for an element in the list
int searchElement(struct Node **head, int data) {
    struct Node* temp = *head;
    int position = 0;
    while(temp != NULL) {
        if(temp->data == data) {
            return position;
        }
        temp = temp->next;
        position++;
    }
    return -1;
}
// Function to delete a node at a specified position
void deleteElement(struct Node** head, int position) {
    if (*head == NULL) {
        printf("Error: List is empty.\n");
        return;
    }
    if (position < 0) {
        printf("Error: Invalid position.\n");
        return;
    }
    struct Node* temp = *head;
    if (position == 0) {
        *head = temp->next;
        free(temp);
        return;
    }
    struct Node* previous = NULL;
    int currentPos = 0;
    while (temp != NULL && currentPos < position) {

```

```

        previous = temp;
        temp = temp->next;
        currentPos++;
    }
    if (temp == NULL) {
        printf("Error: Invalid position.\n");
        return;
    }
    previous->next = temp->next;
    free(temp);
}
// Function to display the list
void displayList(struct Node* head) {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }
    printf("Linked List: ");
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

```

OUTPUT:

Singly Linked List Operations Menu:

1. Insert at End
2. Insert at Position
3. Search Element
4. Delete Element
5. Display List
6. Exit

Enter your choice: 1

Enter data to insert at end: 10

Singly Linked List Operations Menu:

1. Insert at End
2. Insert at Position
3. Search Element
4. Delete Element
5. Display List
6. Exit

Enter your choice: 2

Enter data to insert: 20

Enter position to insert (0-based index): 1

Singly Linked List Operations Menu:

1. Insert at End
2. Insert at Position
3. Search Element
4. Delete Element

5. Display List

6. Exit

Enter your choice: 5

Linked List: 10 -> 20 -> NULL

Singly Linked List Operations Menu:

1. Insert at End

2. Insert at Position

3. Search Element

4. Delete Element

5. Display List

6. Exit

Enter your choice: 3

Enter element to search: 20

Element found at position: 1

Singly Linked List Operations Menu: 1.

Insert at End

2. Insert at Position

3. Search Element

4. Delete Element

5. Display List

6. Exit

Enter your choice: 4

Enter position to delete (0-based index): 1

Singly Linked List Operations Menu:

1. Insert at End

2. Insert at Position

3. Search Element

4. Delete Element

5. Display List

6. Exit

Enter your choice: 5

Linked List: 10 -> NULL

Singly Linked List Operations Menu:

1. Insert at End

2. Insert at Position

3. Search Element

4. Delete Element

5. Display List

6. Exit

Enter your choice: 6